PONTIFICIA UNIVERSIDAD JAVERIANA
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas
Arquitectura de Software

Laboratorio 1

Implementación de un monolito con patrón MVC y DAO

personapi-dotnet

Presentado por:
Miguel Laiton
Sebastián Sánchez Olaya
Angy Lorena Bautista

Profesor:
Andrés Sánchez

Bogotá D.C., octubre de 2026

Tabla de contenido

1. Introducción 5
1.1 Stack tecnológico 5
2. Marco conceptual 6
2.1 Arquitectura monolítica 6
2.2 Patrón Modelo-Vista-Controlador (MVC) 6
2.3 Patrón Data Access Object (DAO) 6
2.4 REST 6
2.5 Swagger y OpenAPI 3 7
2.6 ORM y Entity Framework Core 7
2.7 SQL Server 7
2.8 Inyección de dependencias 7
3. Diseño 8
3.1 Requerimientos 8
3.2 Modelo de datos 8
3.3 Arquitectura 9
3.4 Estructura del proyecto 12
3.5 Interfaces DAO 13
3.6 Endpoints REST 13
3.7 Vistas MVC 14
3.8 Decisiones de diseño 14
4. Procedimiento 16
4.1 Creación del repositorio 16
4.2 Instalación de SQL Server y SSMS 16
4.3 Creación de la base de datos y las tablas 17
4.4 Instalación de Visual Studio 2022 19
4.5 Clonación y creación del proyecto 19
4.6 Paquetes NuGet 21
4.7 Generación de las entidades 21
4.8 Interfaces y repositorios 21
4.9 Controladores y vistas 22
4.10 Despliegue y pruebas 22

4.11 Push y TAG.....27
5. Conclusiones y lecciones aprendidas 29
5.1 Conclusiones.....29
5.2 Lecciones aprendidas.....29
6. Referencias 30

1. Introducción

Este laboratorio consiste en construir una aplicación monolítica en ASP.NET Core que hace el CRUD (crear, consultar, actualizar y eliminar) de un modelo de datos de personas, profesiones, estudios y teléfonos. La aplicación se organiza con el patrón MVC para separar las vistas, los controladores y el modelo, y con el patrón DAO para que el acceso a la base de datos quede aislado en repositorios.

Además de las vistas web, la aplicación expone una API REST con los mismos servicios, documentada con Swagger. Los datos se guardan en SQL Server y el acceso se hace con Entity Framework Core.

El código fuente, los scripts de la base de datos y el README con los pasos de instalación están en el repositorio:

https://github.com/AnGISIta12/personapi-dotnet

[Cambiar por la URL del TAG cuando se cree (ej. .../tree/v1.0.0)]

1.1 Stack tecnológico

Componente	Tecnología	Versión
Framework	ASP.NET Core MVC	.NET 7
Base de datos	Microsoft SQL Server	2022 (o 2019 Express)
ORM	Entity Framework Core	7.0.20
Estilo de API	REST sobre HTTP con JSON	N/A
Documentación de API	Swagger 3 / OpenAPI 3 (Swashbuckle.AspNetCore)	6.5.0
IDE	Visual Studio Community	2022
Control de versiones	Git y GitHub	N/A

Tabla 1. *Tecnologías usadas en el laboratorio.*

El enunciado pide .NET 7, pero el procedimiento del laboratorio menciona .NET 6.0. Se trabajó con .NET 7 porque es lo que aparece en el stack. El código no usa nada exclusivo de .NET 7, así que pasar a .NET 6 solo implica cambiar el framework en el archivo del proyecto y usar los paquetes 6.x.

2. Marco conceptual

2.1 Arquitectura monolítica

Una aplicación monolítica es la que se construye y se despliega como una sola unidad. La interfaz, la lógica de negocio y el acceso a datos viven en el mismo proyecto y corren en el mismo proceso. Es el estilo más directo para empezar, porque se compila y se despliega una sola cosa, y los componentes se comunican con llamadas a métodos dentro del mismo proceso.

Su desventaja aparece cuando la aplicación crece. Cualquier cambio obliga a desplegar todo de nuevo, no se puede escalar una parte por separado y, si no hay disciplina, el código se acopla. Por eso dentro del monolito conviene organizar el código en capas, que es justo lo que hacen MVC y DAO en este laboratorio.

2.2 Patrón Modelo-Vista-Controlador (MVC)

MVC divide una aplicación con interfaz de usuario en tres responsabilidades. El modelo contiene los datos y las reglas del dominio. La vista se encarga de mostrar esos datos. El controlador recibe las peticiones del usuario, decide qué hacer con el modelo y escoge qué vista devolver. El patrón viene de Smalltalk-80 (Krasner y Pope, 1988) y hoy es la base de frameworks web como ASP.NET Core MVC, Spring MVC o Ruby on Rails.

En ASP.NET Core MVC los controladores son clases que heredan de Controller, cada método público es una acción que responde a una ruta, y las vistas son archivos Razor (.cshtml) que mezclan HTML con C#. El framework hace el enlace de modelos (model binding), es decir, convierte los campos de un formulario o el JSON de una petición en objetos C#, y valida esos objetos antes de que lleguen a la acción.

2.3 Patrón Data Access Object (DAO)

El patrón DAO separa la lógica de acceso a datos del resto de la aplicación. Se define una interfaz con las operaciones que se pueden hacer sobre una entidad (por ejemplo, listar, buscar por llave, crear, actualizar y eliminar) y una clase que la implementa con la tecnología de persistencia que se esté usando. El resto del código solo conoce la interfaz (Alur, Crupi y Malks, 2003).

Esto tiene dos ventajas. Si mañana se cambia el motor de base de datos o el ORM, solo cambian las clases que implementan las interfaces. Y como los controladores reciben la interfaz por inyección de dependencias, es fácil reemplazar el repositorio real por uno falso para hacer pruebas. En .NET este patrón casi siempre se conoce como patrón repositorio, que es una variante muy parecida descrita por Fowler (2002).

2.4 REST

REST (Representational State Transfer) es un estilo arquitectónico para sistemas distribuidos que propuso Roy Fielding en su tesis doctoral (Fielding, 2000). En una API REST cada recurso se identifica con una URL y se manipula con los métodos estándar de HTTP: GET para consultar, POST para crear, PUT para actualizar y DELETE para eliminar. La comunicación es sin estado, o sea que cada petición trae toda la información que el servidor necesita para atenderla.

Los códigos de estado HTTP indican el resultado de la operación. En este laboratorio se usan 200 (OK), 201 (Created), 204 (No Content), 400 (Bad Request), 404 (Not Found) y 409 (Conflict).

2.5 Swagger y OpenAPI 3

OpenAPI es una especificación para describir APIs REST en un archivo JSON o YAML: qué rutas existen, qué métodos aceptan, qué parámetros reciben y qué responden. Swagger es el conjunto de herramientas que nació con esa especificación; la versión 3 corresponde a OpenAPI 3.0. Swagger UI lee ese archivo y genera una página web donde se pueden ver y probar los endpoints.

En .NET se usa la librería Swashbuckle.AspNetCore, que genera el documento OpenAPI a partir de los controladores y lo publica en /swagger.

2.6 ORM y Entity Framework Core

Un ORM (Object-Relational Mapping) traduce entre las tablas de una base de datos relacional y las clases de un lenguaje orientado a objetos. Con un ORM se trabaja con objetos y colecciones, y el ORM genera el SQL.

Entity Framework Core es el ORM oficial de .NET. La clase principal es el DbContext, que representa una sesión con la base de datos y tiene un DbSet por cada tabla. EF Core permite trabajar de dos formas: code first, donde las tablas se generan a partir de las clases, y database first, donde las clases se generan a partir de una base que ya existe. En este laboratorio se usó database first con el comando Scaffold-DbContext.

2.7 SQL Server

Microsoft SQL Server es un sistema gestor de bases de datos relacionales. La edición Express es gratuita y es suficiente para desarrollo. Se administra con SQL Server Management Studio (SSMS). Hay que tener en cuenta que su dialecto de SQL (T-SQL) no es igual al de MySQL: no tiene el tipo ENUM, no usa INT(n) y TEXT está obsoleto, por lo que el modelo del enunciado se tuvo que adaptar (ver sección 3.2).

2.8 Inyección de dependencias

ASP.NET Core trae un contenedor de inyección de dependencias. En Program.cs se registra qué clase concreta se usa para cada interfaz y el framework se encarga de crear los objetos y pasarlos por el constructor. Así un controlador recibe un IPersonaRepository sin saber que por debajo hay un PersonaRepository con EF Core. Los repositorios se registran como Scoped, es decir, se crea una instancia por cada petición HTTP.

3. Diseño

3.1 Requerimientos

El enunciado pide endpoints y vistas para el CRUD de cuatro entidades. En concreto:

- Crear, listar, consultar por llave, actualizar y eliminar registros de profesion, persona, estudios y telefono.
- Hacerlo desde vistas web (MVC) y desde una API REST.
- Documentar la API con Swagger 3.
- Acceder a la base de datos solo a través de interfaces DAO y sus repositorios.
- Entregar el código con un TAG en git, un README y los scripts DDL y DML.

3.2 Modelo de datos

El modelo tiene cuatro tablas. La tabla estudios es la intermedia de una relación muchos a muchos entre persona y profesión, y su llave primaria es compuesta (id_prof, cc_per). Una persona puede tener varios teléfonos y cada teléfono tiene un solo dueño.

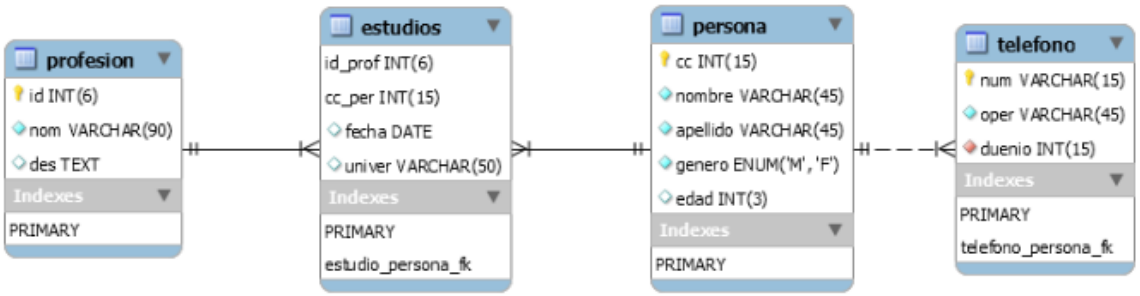


Figura 1. *Modelo entidad relación de persona_db.*

El diagrama del enunciado usa tipos de MySQL. Para SQL Server se hicieron estos cambios:

Tabla.columna	Enunciado	SQL Server	Motivo
profesion.des	TEXT	VARCHAR(MAX)	TEXT está obsoleto en SQL Server
persona.genero	ENUM('M','F')	CHAR(1) + CHECK	SQL Server no tiene ENUM
Columnas INT(n)	INT(6), INT(15), INT(3)	INT	SQL Server no maneja ancho en INT
Llaves	PRIMARY / *_fk	PK_<tabla> y FK con el mismo nombre	Se conservaron los nombres de las FK

Tabla 2. *Adaptación de tipos del modelo a SQL Server.*

INT llega hasta 2.147.483.647, así que las cédulas de prueba se escogieron por debajo de ese valor. Si se necesitan cédulas más largas habría que cambiar cc, cc_per y duenio a BIGINT.

3.3 Arquitectura

La aplicación es un solo proyecto ASP.NET Core organizado en capas. Hay dos puntos de entrada que comparten todo lo demás: los controladores MVC, que devuelven vistas HTML, y los controladores API, que devuelven JSON. Ninguno de los dos toca el DbContext. Ambos dependen de las interfaces DAO, y solo los repositorios usan el PersonaDbContext para hablar con SQL Server.

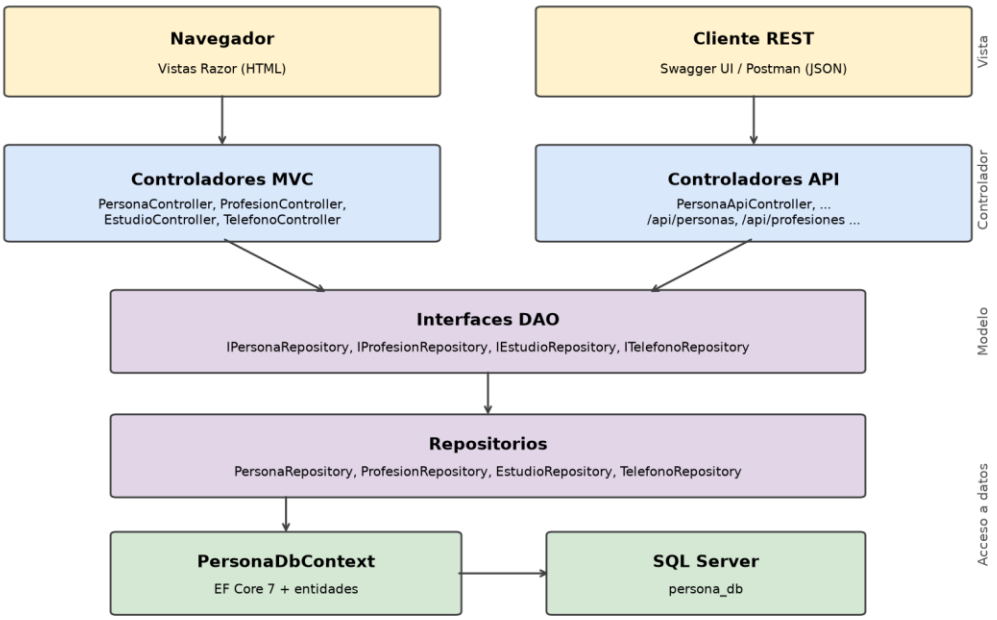


Figura 2. *Capas de la aplicación y dependencias entre ellas.*

Además de las capas, la arquitectura se describe con el modelo C4, que la muestra en cuatro niveles de detalle: contexto, contenedores, componentes y código. Cada nivel acerca la vista al anterior.

El diagrama de contexto muestra el sistema como una sola caja y quién lo usa: el usuario de la aplicación, que trabaja desde el navegador con las vistas, y el cliente de la API, que consume el CRUD por REST desde Swagger UI o un cliente HTTP.

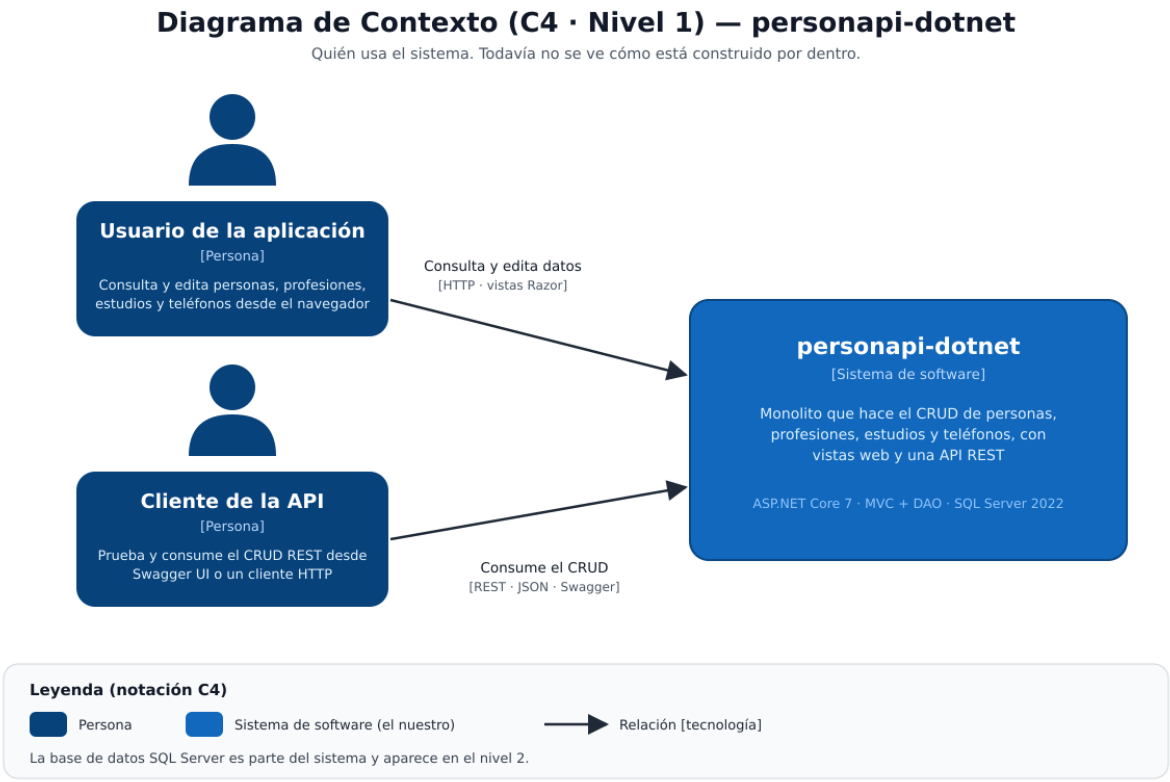


Figura 3. *Diagrama de contexto (C4, nivel 1).*

El diagrama de contenedores separa lo que se despliega: una aplicación web ASP.NET Core 7 (puerto 5142 en desarrollo) que sirve las vistas Razor y la API REST, y la base persona_db en SQL Server 2022. Es un monolito: un solo proceso y una sola base de datos.

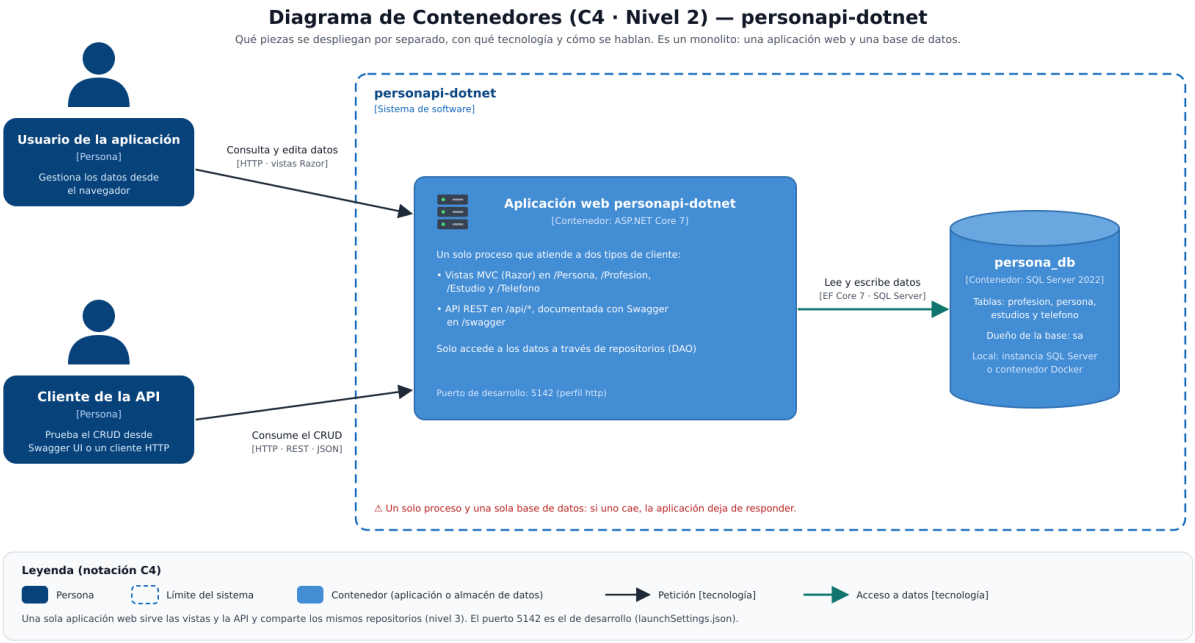


Figura 4. *Diagrama de contenedores (C4, nivel 2).*

El diagrama de componentes abre la aplicación web: controladores MVC y API, vistas Razor, interfaces DAO, repositorios, PersonaDbContext y Swagger. El camino de una petición es controlador, interfaz, repositorio, DbContext y SQL Server.

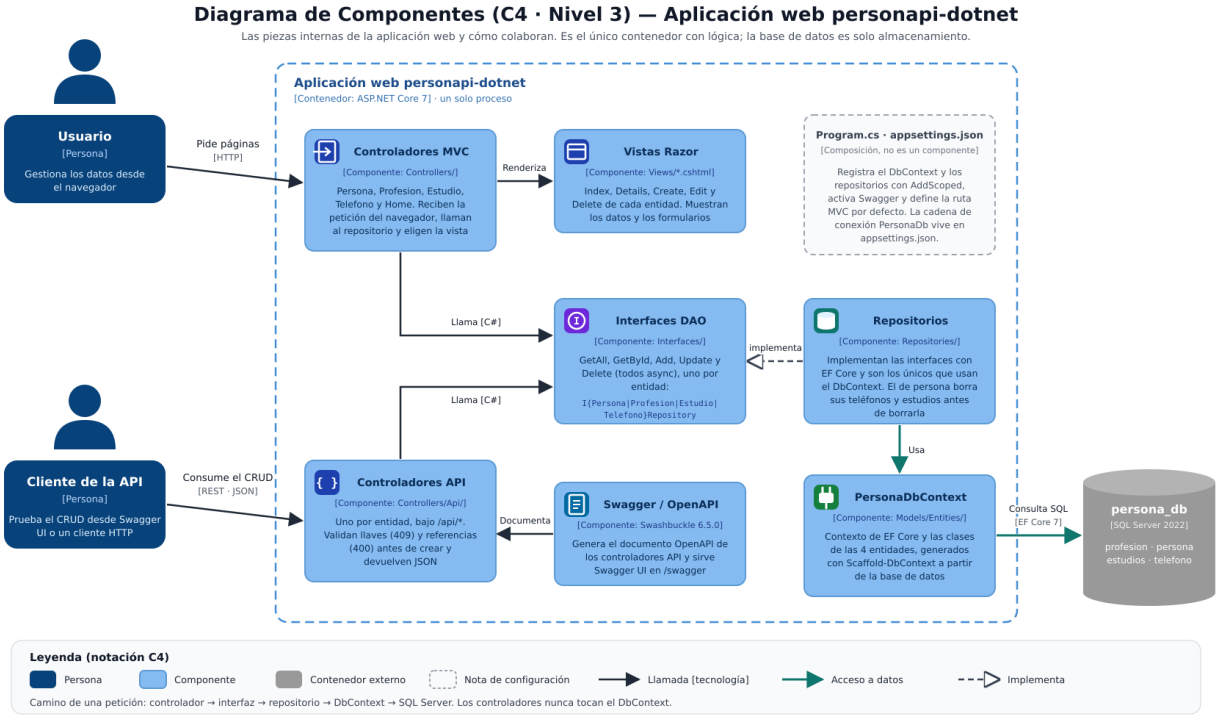


Figura 5. *Diagrama de componentes (C4, nivel 3).*

El diagrama de código baja a los tipos de C# y usa la entidad Estudio como ejemplo: EstudioApiController depende de interfaces, y EstudioRepository implementa IEstudioRepository y usa el PersonaDbContext. Persona, Profesión y Teléfono repiten la misma forma.

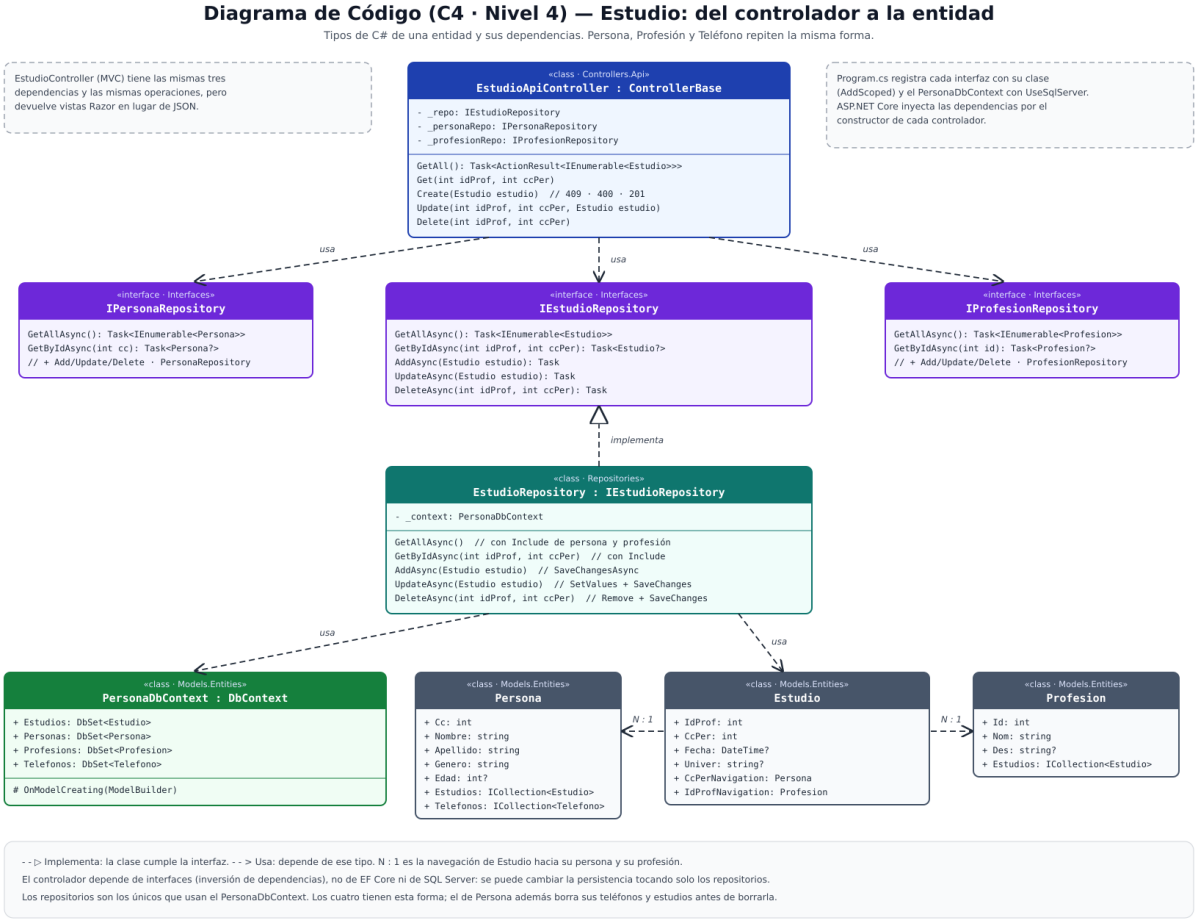


Figura 6. *Diagrama de código (C4, nivel 4).*

3.4 Estructura del proyecto

```
personapi-dotnet/
├── database/
│   ├── ddl.sql
│   └── dml.sql
├── personapi-dotnet.sln
├── README.md
└── personapi-dotnet/
    ├── Controllers/
    │   ├── Api/              (controladores REST)
    │   ├── PersonaController.cs
    │   ├── ProfesionController.cs
    │   ├── EstudioController.cs
    │   └── TelefonoController.cs
```

```
├──── Interfaces/          (interfaces DAO)
├──── Repositories/        (implementación DAO)
├──── Models/Entities/     (entidades y DbContext)
├──── Views/               (Persona, Profesion, Estudio, Telefono)
├──── appsettings.json
└──── Program.cs
```

3.5 Interfaces DAO

Cada entidad tiene su interfaz con las mismas cinco operaciones. Lo único que cambia es el tipo de la llave:

Interfaz	Llave	Métodos
IPersonaRepository	int cc	GetAllAsync, GetByIdAsync(cc), AddAsync, UpdateAsync, DeleteAsync(cc)
IProfesionRepository	int id	GetAllAsync, GetByIdAsync(id), AddAsync, UpdateAsync, DeleteAsync(id)
ITelefonoRepository	string num	GetAllAsync, GetByIdAsync(num), AddAsync, UpdateAsync, DeleteAsync(num)
IEstudioRepository	int idProf, int ccPer	GetAllAsync, GetByIdAsync(idProf, ccPer), AddAsync, UpdateAsync, DeleteAsync(idProf, ccPer)

Tabla 3. *Interfaces DAO y sus operaciones.*

Ejemplo de una de las interfaces:

```
public interface IPersonaRepository
{
    Task<IEnumerable<Persona>> GetAllAsync();
    Task<Persona?> GetByIdAsync(int cc);
    Task AddAsync(Persona persona);
    Task UpdateAsync(Persona persona);
    Task DeleteAsync(int cc);
}
```

Los métodos son asíncronos (devuelven Task) porque EF Core trae versiones asíncronas de las consultas y así el servidor no bloquea un hilo mientras espera a la base de datos.

3.6 Endpoints REST

Método	Ruta	Descripción	Respuestas
GET	/api/personas	Lista todas las personas	200
GET	/api/personas/{cc}	Consulta una persona	200, 404
POST	/api/personas	Crea una persona	201, 400, 409
PUT	/api/personas/{cc}	Actualiza una persona	204, 400, 404

DELETE	/api/personas/{cc}	Elimina una persona con sus teléfonos y estudios	204, 404
GET / POST	/api/profesiones	Lista o crea profesiones	200 / 201, 409
GET / PUT / DELETE	/api/profesiones/{id}	Consulta, actualiza o elimina una profesión	200, 204, 404
GET / POST	/api/telefonos	Lista o crea teléfonos	200 / 201, 400, 409
GET / PUT / DELETE	/api/telefonos/{num}	Consulta, actualiza o elimina un teléfono	200, 204, 400, 404
GET / POST	/api/estudios	Lista o crea estudios	200 / 201, 400, 409
GET / PUT / DELETE	/api/estudios/{idProf}/{ccPer}	Consulta, actualiza o elimina un estudio	200, 204, 404

Tabla 4. *Endpoints de la API REST.*

El 409 sale cuando se intenta crear un registro con una llave que ya existe. El 400 sale cuando la llave de la URL no coincide con la del cuerpo de la petición, o cuando un teléfono o un estudio hace referencia a una persona o profesión que no existe.

3.7 Vistas MVC

Cada entidad tiene un controlador MVC con las acciones Index, Details, Create, Edit y Delete, y una vista Razor para cada una. Las vistas siguen la estructura que genera Visual Studio con la plantilla de controlador con vistas, pero los controladores se modificaron para usar el repositorio en vez del DbContext.

Ruta	Qué muestra
/Persona	Tabla de personas con enlaces para editar, ver detalles y eliminar
/Profesion	Tabla de profesiones
/Telefono	Tabla de teléfonos con el nombre del dueño; el dueño se escoge de una lista
/Estudio	Tabla de estudios con el nombre de la persona y de la profesión; ambas se escogen de listas
/swagger	Swagger UI con la documentación de la API

Tabla 5. *Vistas de la aplicación.*

3.8 Decisiones de diseño

Durante la implementación salieron algunos problemas que obligaron a tomar decisiones:

Llave compuesta de estudios. La plantilla de Visual Studio asume que cada entidad tiene un solo id, así que para estudios las rutas y los formularios se cambiaron para mandar idProf y ccPer. En la API la ruta queda /api/estudios/{idProf}/{ccPer}. Por la misma razón, en las vistas de edición la llave se muestra pero no se deja modificar; para cambiarla hay que borrar el registro y crearlo otra vez.

Ciclos al serializar a JSON. Las entidades que genera el Scaffold tienen propiedades de navegación en los dos sentidos (una persona tiene teléfonos y cada teléfono tiene su persona). Al devolverlas como JSON eso produce un ciclo. Se les puso [JsonIgnore] para que la API devuelva solo los campos de la tabla y para que Swagger no pida objetos anidados al crear un registro.

Validación de las propiedades de navegación. Con los tipos de referencia no anulables de C#, ASP.NET Core trata las propiedades de navegación como obligatorias y el formulario de teléfono o estudio fallaba porque no venía la persona completa. Se les agregó [ValidateNever] para que la validación las ignore.

Borrado de registros relacionados. Las llaves foráneas no tienen borrado en cascada, así que borrar una persona con teléfonos daba error. El repositorio de persona borra primero sus teléfonos y estudios, y el de profesión borra los estudios que la usan.

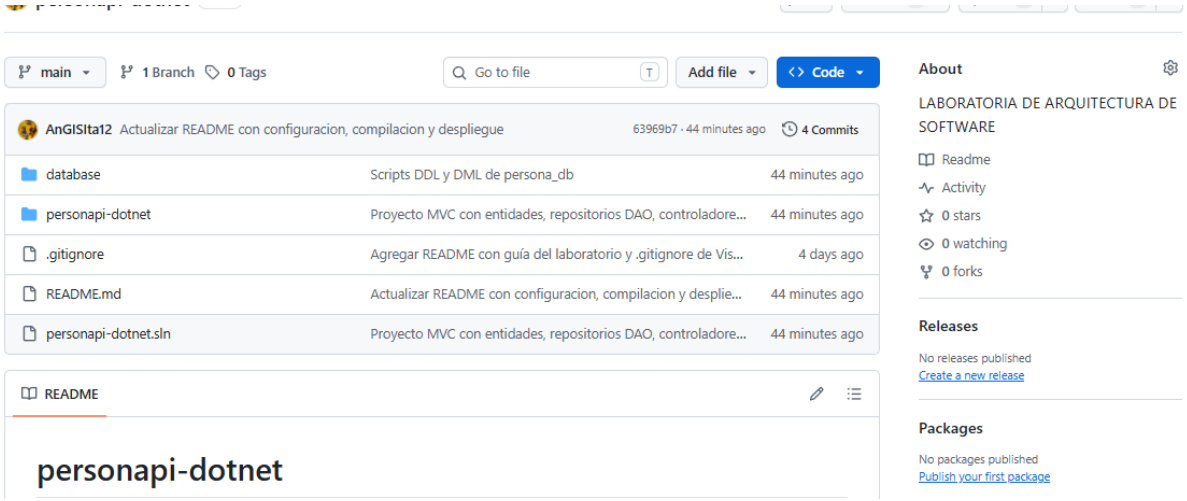
Actualización. El repositorio busca el registro actual y le copia los valores nuevos con SetValues, en lugar de llamar Update sobre el objeto que llega. Así no hay conflicto cuando el controlador ya cargó la entidad para verificar que existe.

4. Procedimiento

Estos son los pasos que se siguieron, en el orden del enunciado.

4.1 Creación del repositorio

Se creó el repositorio público personapi-dotnet en GitHub, con un README y el .gitignore de Visual Studio para no subir las carpetas bin, obj y .vs.



4.2 Instalación de SQL Server y SSMS

El enunciado pide SQL Server Express en modo Básico, que deja la instancia con el nombre localhost\SQLEXPRESS. En este laboratorio se usó en su lugar SQL Server 2022 en un contenedor de Docker, accesible en localhost,1433 con el usuario sa; para el resto del procedimiento funciona igual. Luego se instaló SQL Server Management Studio para administrarlo. Con esta instancia, la cadena de conexión de la sección 4.7 usa Server=localhost,1433 y el usuario sa en lugar de localhost\SQLEXPRESS con autenticación de Windows.

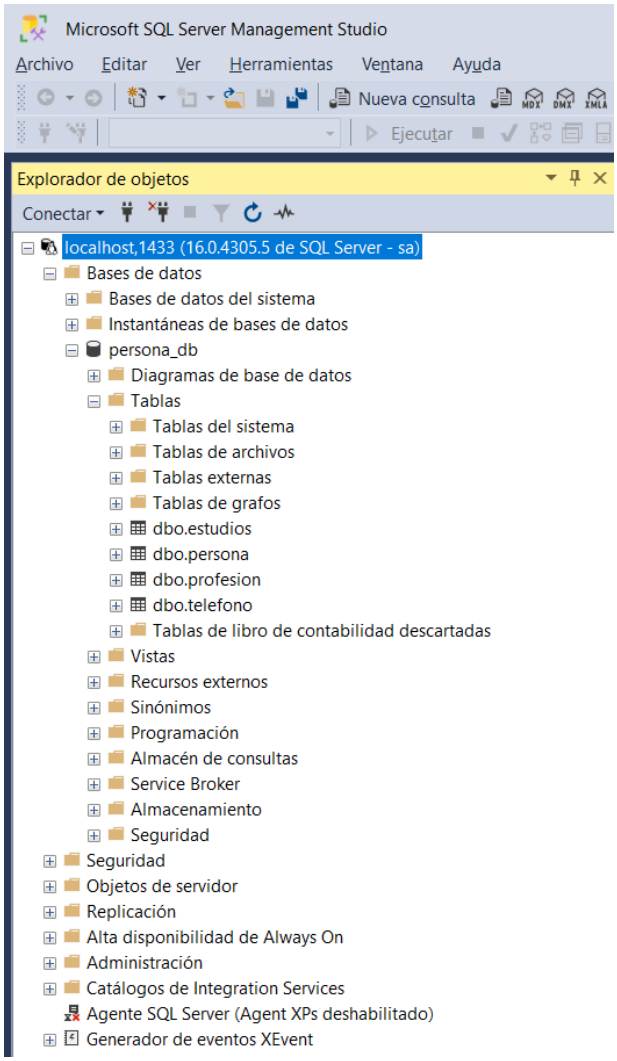


Figura 8. *SQL Server Management Studio conectado a la instancia (localhost,1433), con la base persona_db y sus cuatro tablas.*

4.3 Creación de la base de datos y las tablas

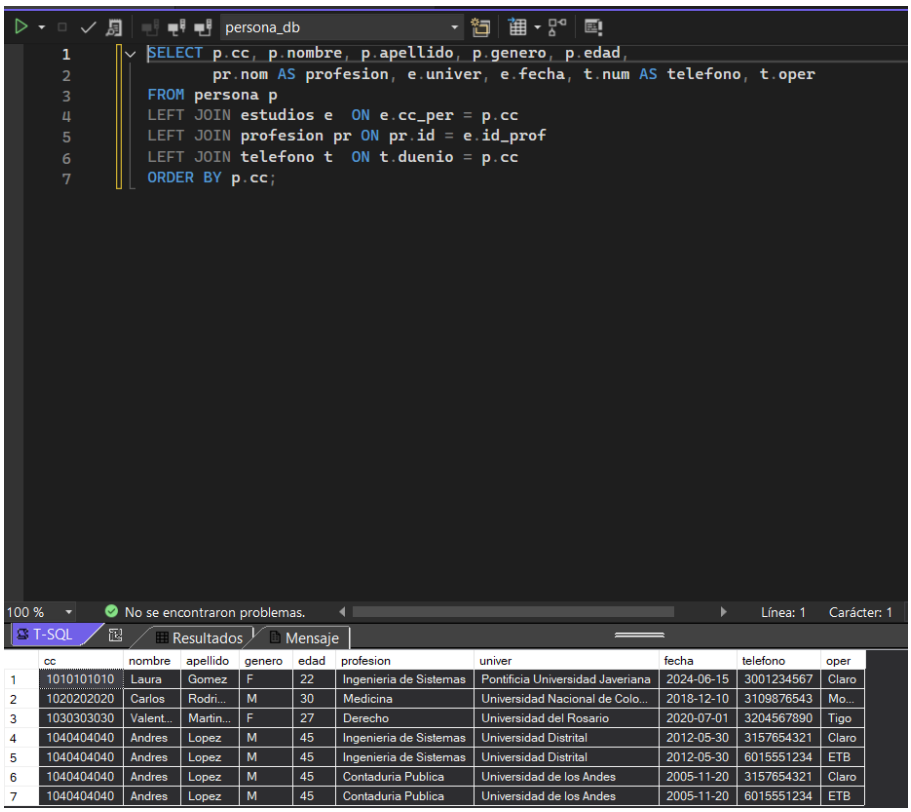
En SSMS se ejecutó el script database/ddl.sql. Este crea la base persona_db, le asigna como propietario al usuario sa y crea las cuatro tablas con sus llaves:

```
CREATE DATABASE persona_db;
GO
ALTER AUTHORIZATION ON DATABASE::persona_db TO sa;
GO
USE persona_db;
GO
CREATE TABLE persona (
   cc       INT         NOT NULL,
   nombre   VARCHAR(45) NOT NULL,
   apellido VARCHAR(45) NOT NULL,
   genero   CHAR(1)     NOT NULL,
```

```
    edad     INT         NULL,
    CONSTRAINT PK_persona PRIMARY KEY (cc),
    CONSTRAINT CK_persona_genero CHECK (genero IN ('M', 'F'))
);
-- ... profesion, estudios y telefono (ver database/ddl.sql)
```

Después se ejecutó database/dml.sql, que inserta cuatro profesiones, cuatro personas, cinco estudios y cinco teléfonos de prueba.





	cc	nombre	apellido	genero	edad	profesion	univer	fecha	telefono	oper
1	1010101010	Laura	Gomez	F	22	Ingenieria de Sistemas	Pontificia Universidad Javeriana	2024-06-15	3001234567	Claro
2	1020202020	Carlos	Rodri...	M	30	Medicina	Universidad Nacional de Colo...	2018-12-10	3109876543	Mo...
3	1030303030	Valent...	Martin...	F	27	Derecho	Universidad del Rosario	2020-07-01	3204567890	Tigo
4	1040404040	Andres	Lopez	M	45	Ingenieria de Sistemas	Universidad Distrital	2012-05-30	3157654321	Claro
5	1040404040	Andres	Lopez	M	45	Ingenieria de Sistemas	Universidad Distrital	2012-05-30	6015551234	ETB
6	1040404040	Andres	Lopez	M	45	Contaduria Publica	Universidad de los Andes	2005-11-20	3157654321	Claro
7	1040404040	Andres	Lopez	M	45	Contaduria Publica	Universidad de los Andes	2005-11-20	6015551234	ETB

Figura 10. *Consulta sobre persona_db que une las cuatro tablas y muestra los datos de prueba.*

4.4 Instalación de Visual Studio 2022

Se instaló Visual Studio Community 2022 con las cargas de trabajo Desarrollo de ASP.NET y web, Almacenamiento y procesamiento de datos, Plantillas de proyecto y elementos de .NET Framework y Características avanzadas de ASP.NET.

4.5 Clonación y creación del proyecto

Se clonó el repositorio y dentro de esa carpeta se creó un proyecto con la plantilla Aplicación web de ASP.NET Core (Modelo-Vista-Controlador), con el nombre personapi-dotnet, sin autenticación y sin HTTPS.

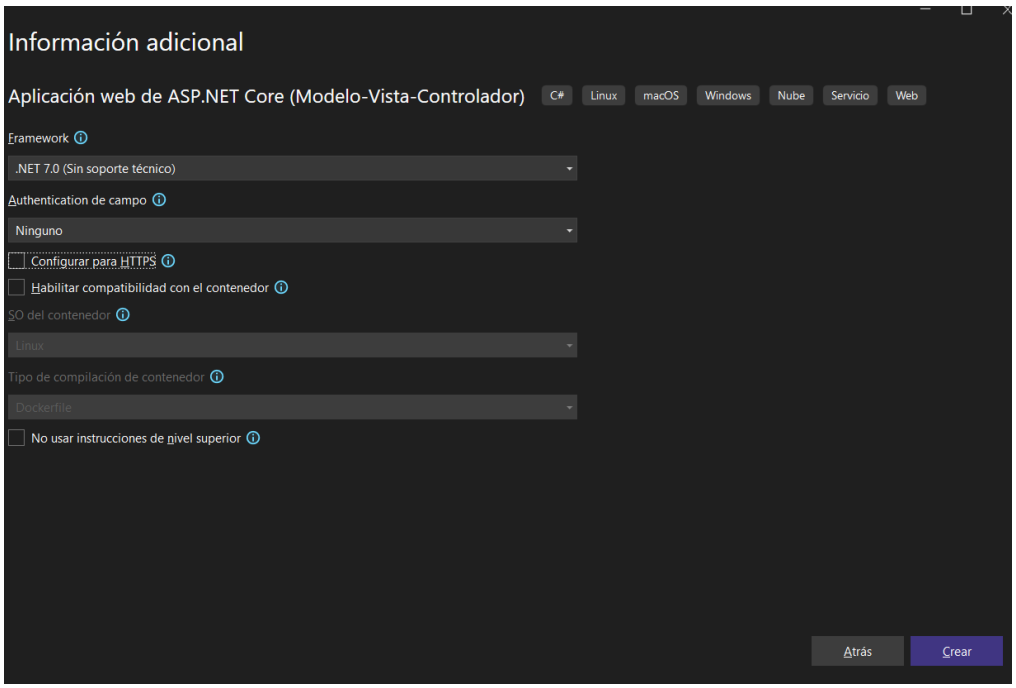


Figura 11. *Configuración del nuevo proyecto en Visual Studio (marco .NET 7.0, sin autenticación y sin HTTPS).*

Desde Ver > Explorador de objetos de SQL Server se agregó la conexión a la instancia de SQL Server que corre en Docker (localhost,1433, usuario sa) para comprobar que Visual Studio veía la base persona_db.

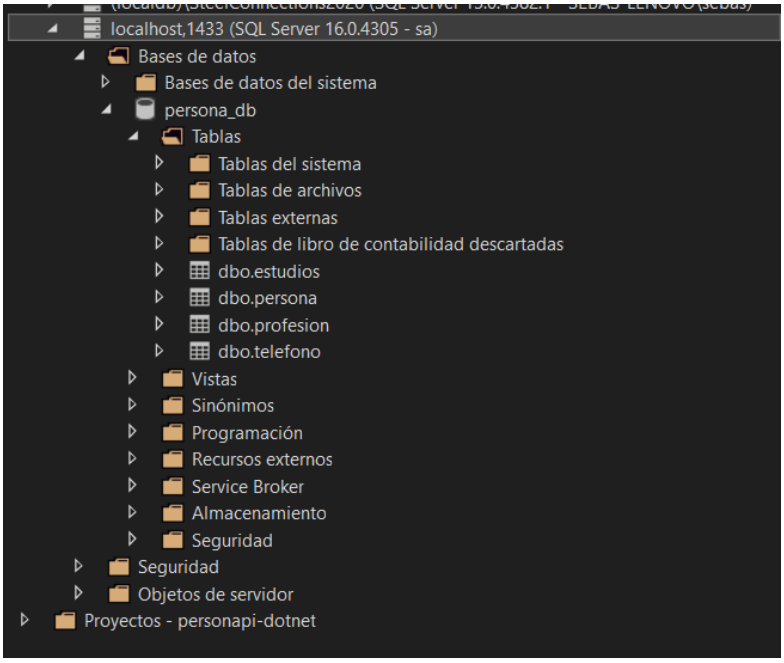
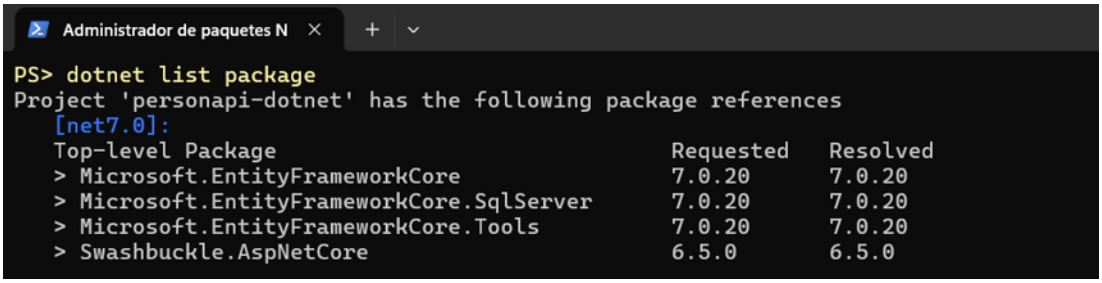


Figura 12. *Explorador de objetos de SQL Server con la base persona_db y sus cuatro tablas.*

4.6 Paquetes NuGet

En el Explorador de soluciones se hizo clic derecho en Dependencias y se eligió Administrar paquetes NuGet. Desde ahí se instalaron Microsoft.EntityFrameworkCore, Microsoft.EntityFrameworkCore.SqlServer y Microsoft.EntityFrameworkCore.Tools en la versión 7.0.20, y Swashbuckle.AspNetCore 6.5.0 para Swagger.



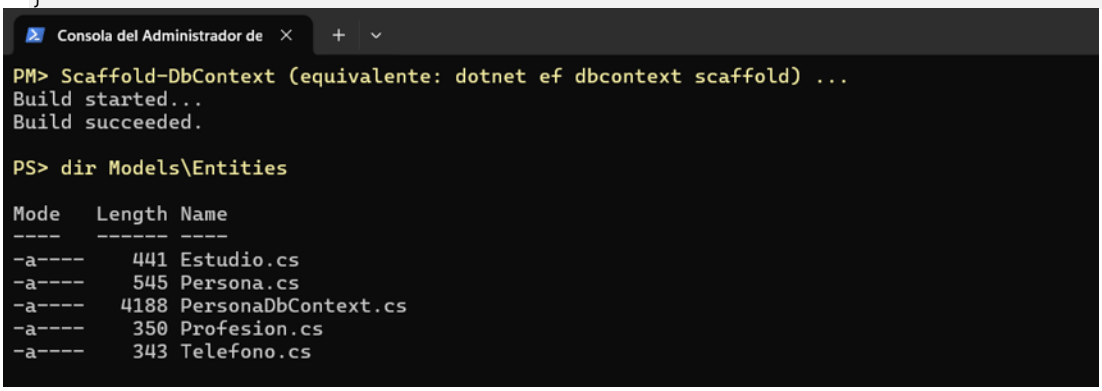
4.7 Generación de las entidades

Se creó la carpeta Models/Entities y en la Consola del Administrador de paquetes se ejecutó:

```
Scaffold-DbContext
"Server=localhost\SQLEXPRESS;Database=persona_db;Trusted_Connection=True;
   TrustServerCertificate=true" Microsoft.EntityFrameworkCore.SqlServer -OutputDir
Models/Entities
```

El comando generó las clases Persona, Profesion, Estudio y Telefono, y el contexto PersonaDbContext. Luego se quitó el método OnConfiguring, que trae la cadena de conexión escrita en el código, y la cadena se pasó a appsettings.json:

```
"ConnectionStrings": {
 "PersonaDb":
"Server=localhost\\SQLEXPRESS;Database=persona_db;Trusted_Connection=True;TrustServerCer
tificate=true"
}
```



4.8 Interfaces y repositorios

En la carpeta Interfaces se creó una interfaz por entidad y en Repositories su implementación con EF Core. Todo se registró en Program.cs junto con el DbContext y Swagger:

```
builder.Services.AddDbContext<PersonaDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("PersonaDb")));

builder.Services.AddScoped<IPersonaRepository, PersonaRepository>();
builder.Services.AddScoped<IProfesionRepository, ProfesionRepository>();
builder.Services.AddScoped<IEstudioRepository, EstudioRepository>();
builder.Services.AddScoped<ITelefonoRepository, TelefonoRepository>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

4.9 Controladores y vistas

Se hicieron dos grupos de controladores. En Controllers/Api quedaron los controladores REST con el atributo [ApiController] y una ruta fija, por ejemplo [Route("api/personas")]. En Controllers quedaron los controladores MVC con sus vistas en Views. En los dos casos el constructor recibe la interfaz del repositorio:

```
public class PersonaController : Controller
{
    private readonly IPersonaRepository _repo;

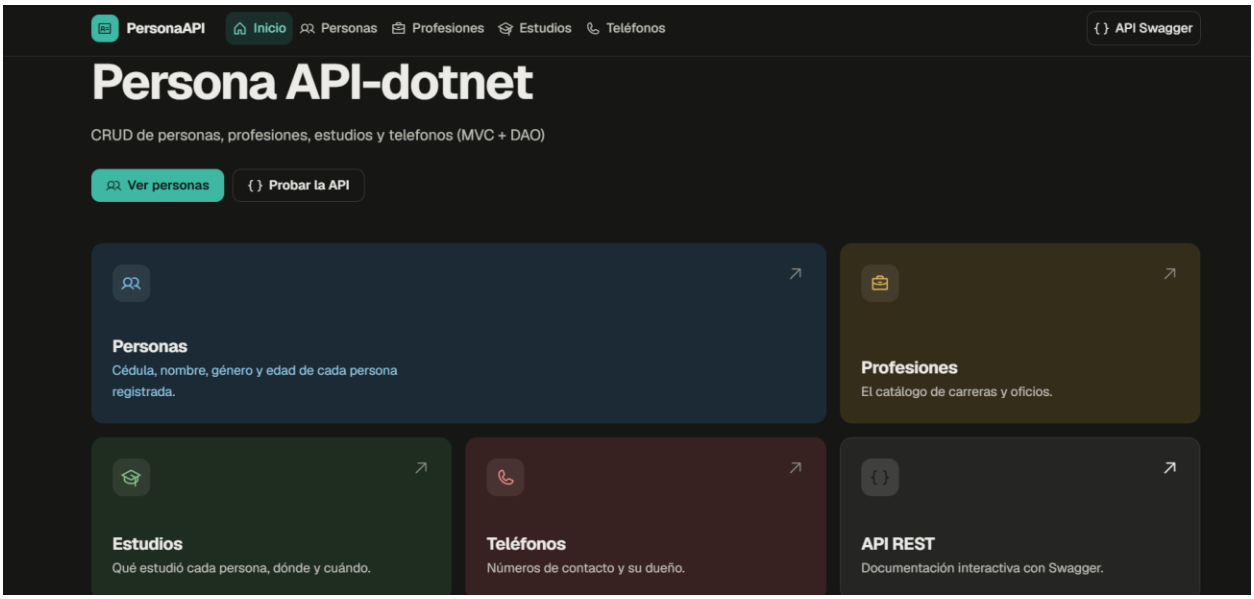
    public PersonaController(IPersonaRepository repo)
    {
        _repo = repo;
    }

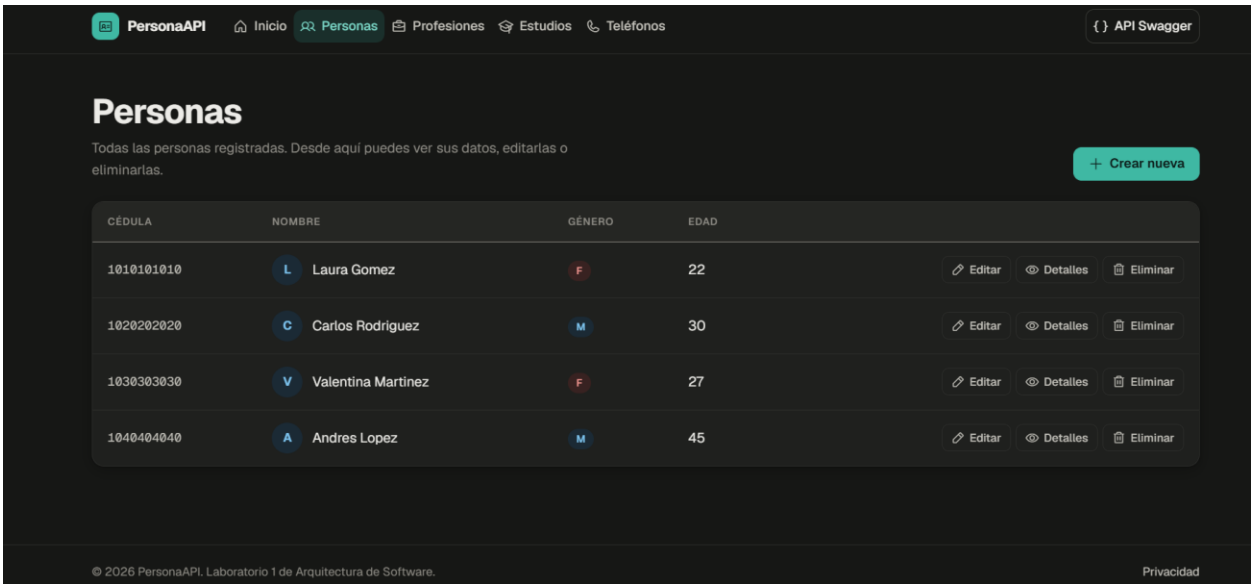
    public async Task<IActionResult> Index()
    {
        return View(await _repo.GetAllAsync());
    }
    ...
}
```

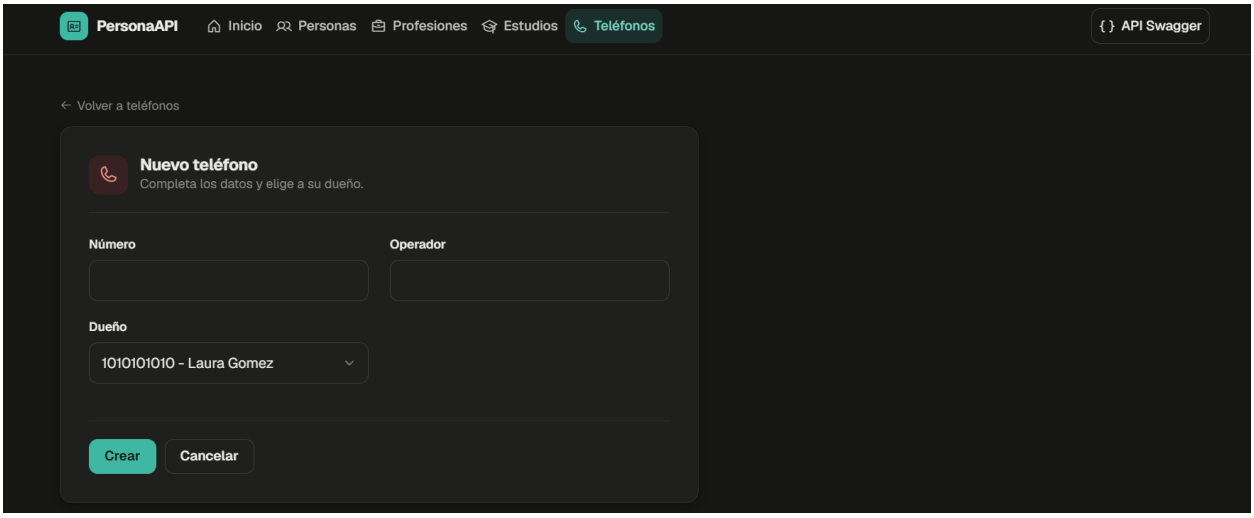
En el menú principal (_Layout.cshtml) se agregaron enlaces a Personas, Profesiones, Estudios, Teléfonos y Swagger.

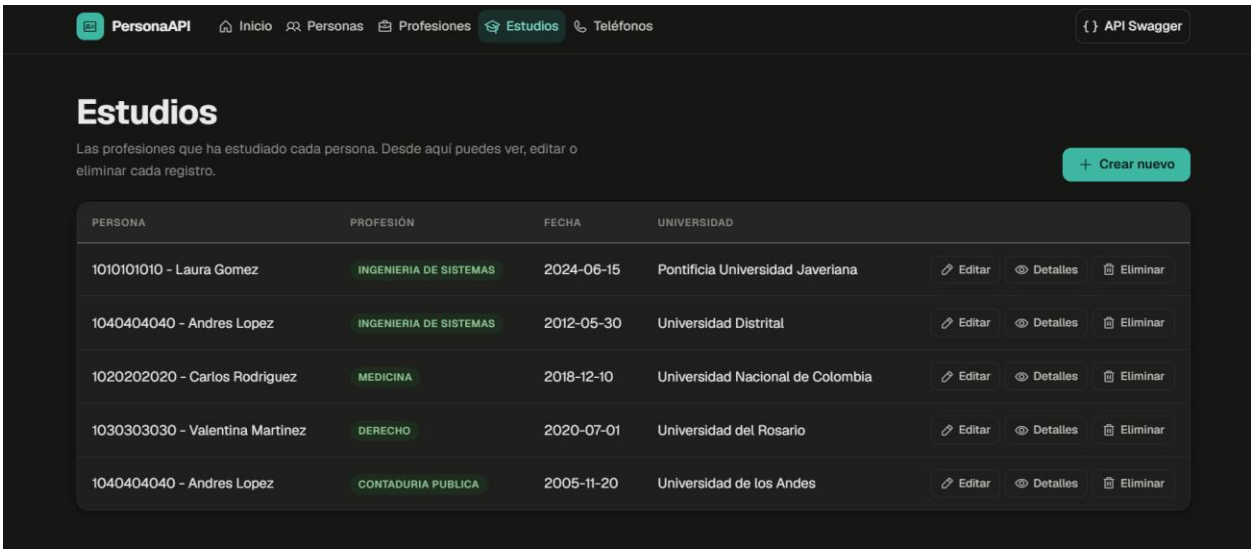
4.10 Despliegue y pruebas

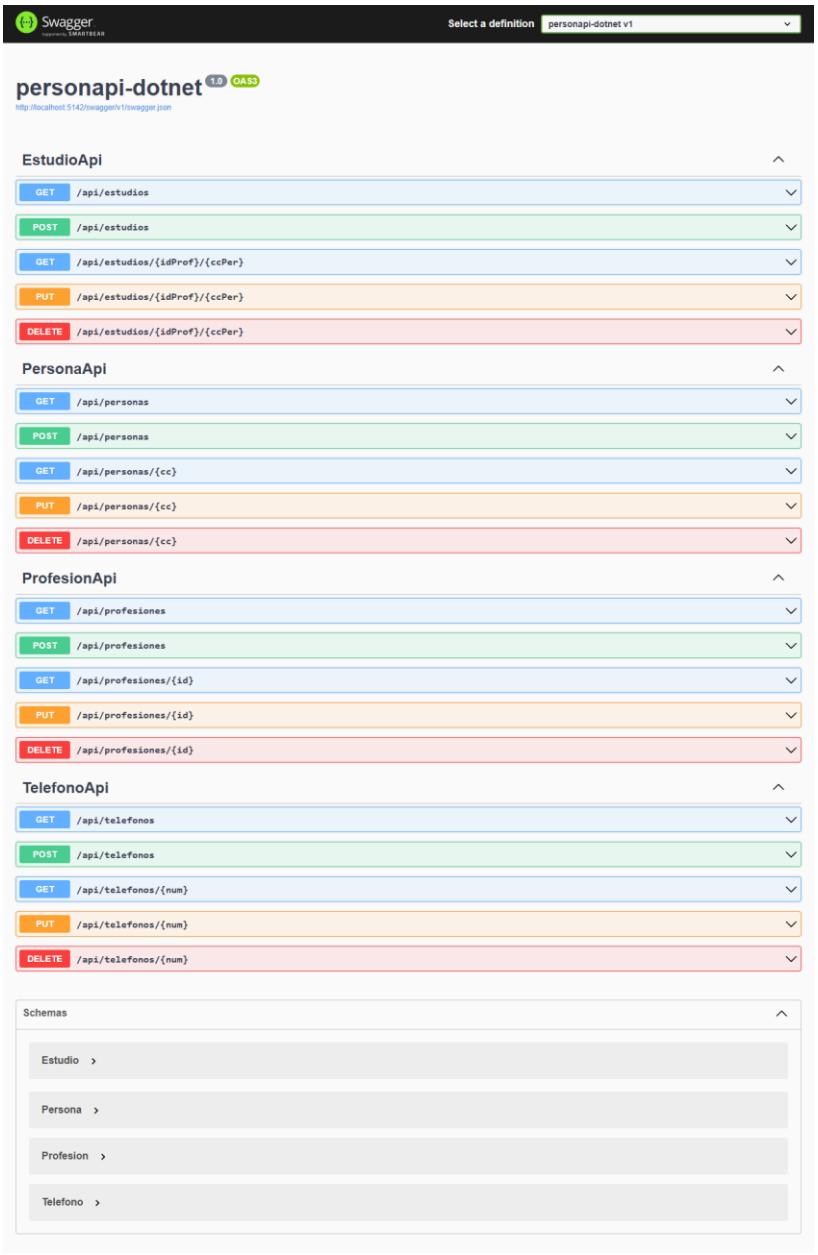
La aplicación se ejecuta con F5 desde Visual Studio o con dotnet run, y queda en http://localhost:5142. Se probó el CRUD completo de las cuatro entidades desde las vistas y desde Swagger, creando los datos en el orden profesión, persona y luego estudios y teléfonos, por las llaves foráneas.

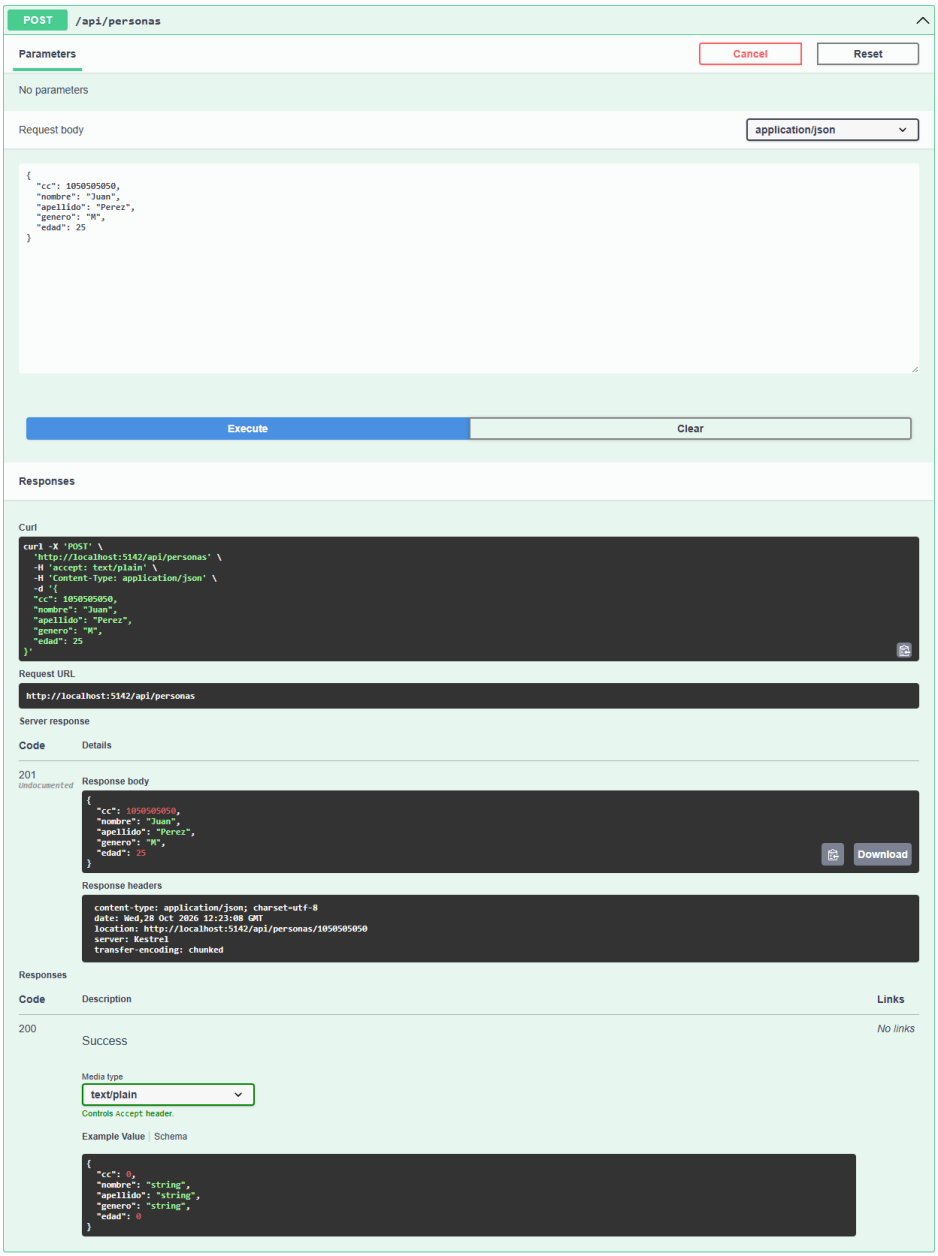

PersonaAPI
Inicio
Personas
Profesiones
Estudios
Teléfonos
{ } API Swagger
Persona API-dotnet
CRUD de personas, profesiones, estudios y telefonos (MVC + DAO)
Ver personas
{ } Probar la API
Personas
Cédula, nombre, género y edad de cada persona registrada.
Profesiones
El catálogo de carreras y oficios.
Estudios
Qué estudió cada persona, dónde y cuándo.
Teléfonos
Números de contacto y su dueño.
API REST
Documentación interactiva con Swagger.


PersonaAPI
Inicio
Personas
Profesiones
Estudios
Teléfonos
{ } API Swagger
Personas
Todas las personas registradas. Desde aquí puedes ver sus datos, editarlas o eliminarlas.
+ Crear nueva
CÉDULA
NOMBRE
GÉNERO
EDAD
1010101010
Laura Gomez
F
22
Editar
Detalles
Eliminar
1020202020
Carlos Rodriguez
M
30
Editar
Detalles
Eliminar
1030303030
Valentina Martinez
F
27
Editar
Detalles
Eliminar
1040404040
Andres Lopez
M
45
Editar
Detalles
Eliminar
© 2026 PersonaAPI. Laboratorio 1 de Arquitectura de Software.
Privacidad


PersonaAPI
Inicio
Personas
Profesiones
Estudios
Teléfonos
{ } API Swagger
← Volver a teléfonos
Nuevo teléfono
Completa los datos y elige a su dueño.
Número
Operador
Dueño
1010101010 - Laura Gomez
Crear
Cancelar


PersonaAPI
Inicio
Personas
Profesiones
Estudios
Teléfonos
{} API Swagger
Estudios
Las profesiones que ha estudiado cada persona. Desde aquí puedes ver, editar o eliminar cada registro.
+ Crear nuevo
PERSONA
PROFESIÓN
FECHA
UNIVERSIDAD
1010101010 - Laura Gomez
INGENIERIA DE SISTEMAS
2024-06-15
Pontificia Universidad Javeriana
Editar
Detalles
Eliminar
1040404040 - Andres Lopez
INGENIERIA DE SISTEMAS
2012-05-30
Universidad Distrital
Editar
Detalles
Eliminar
1020202020 - Carlos Rodriguez
MEDICINA
2018-12-10
Universidad Nacional de Colombia
Editar
Detalles
Eliminar
1030303030 - Valentina Martinez
DERECHO
2020-07-01
Universidad del Rosario
Editar
Detalles
Eliminar
1040404040 - Andres Lopez
CONTADURIA PUBLICA
2005-11-20
Universidad de los Andes
Editar
Detalles
Eliminar


Swagger
Select a definition
personapi-dotnet v1
personapi-dotnet
1.0
OAS3
http://localhost:5142/swagger/v1/swagger.json
EstudioApi
GET /api/estudios
POST /api/estudios
GET /api/estudios/{idProf}/{ccPer}
PUT /api/estudios/{idProf}/{ccPer}
DELETE /api/estudios/{idProf}/{ccPer}
PersonaApi
GET /api/personas
POST /api/personas
GET /api/personas/{cc}
PUT /api/personas/{cc}
DELETE /api/personas/{cc}
ProfesionApi
GET /api/profesiones
POST /api/profesiones
GET /api/profesiones/{id}
PUT /api/profesiones/{id}
DELETE /api/profesiones/{id}
TelefonoApi
GET /api/telefonos
POST /api/telefonos
GET /api/telefonos/{num}
PUT /api/telefonos/{num}
DELETE /api/telefonos/{num}
Schemas
Estudio
Persona
Profesion
Telefono



Para publicar la aplicación fuera de Visual Studio se usa:

```
dotnet publish personapi-dotnet/personapi-dotnet.csproj -c Release -o ./publish
dotnet ./publish/personapi-dotnet.dll --urls http://localhost:5000
```

4.11 Push y TAG

Al terminar se subieron los cambios al repositorio y se creó el TAG de la entrega:

```
git push origin main
git tag -a v1.0.0 -m "Entrega laboratorio 1"
git push origin v1.0.0
```

[Pantallazo pendiente (Figura 21): TAG v1.0.0 en GitHub]

5. Conclusiones y lecciones aprendidas

Este laboratorio mostró que separar las responsabilidades en capas facilita mucho el trabajo: al poner el acceso a datos detrás de interfaces DAO, los controladores MVC y los de la API REST reutilizaron los mismos repositorios y quedaron cortos y fáciles de leer. También se vio que el código generado automáticamente (Scaffold-DbContext y las plantillas de Visual Studio) es un buen punto de partida, pero casi nunca sirve tal cual: hubo que ajustar las propiedades de navegación con [JsonIgnore] y [ValidateNever] para evitar ciclos de serialización y errores de validación, y adaptar a mano las rutas y formularios de la llave compuesta de estudios. Las diferencias entre motores de base de datos (ENUM e INT(n) en MySQL frente a CHECK e INT en SQL Server) y las llaves foráneas sin borrado en cascada obligan a decidir de forma consciente dónde se maneja la integridad, que aquí fue en los repositorios. Por último, dejar la cadena de conexión en appsettings.json (o en variables de entorno) en vez de en el código, y documentar el procedimiento con README, scripts DDL/DML y capturas, permite que otro equipo reproduzca el proyecto sin depender de la máquina de quien lo construyó, algo que quedó claro al correrlo aquí con SQL Server en Docker en lugar de SQLEXPRESS.

5.1 Conclusiones

Separar el acceso a datos en interfaces y repositorios hizo que los controladores quedaran cortos y fáciles de leer. Los controladores MVC y los de la API usan exactamente los mismos repositorios, así que la lógica de acceso a datos se escribió una sola vez y sirvió para las dos formas de entrada.

El enfoque database first con Scaffold-DbContext ahorra mucho trabajo porque genera las entidades y el mapeo a partir de las tablas. Aun así, las clases generadas no siempre sirven tal cual; hubo que ajustarlas para la serialización a JSON y la validación de formularios.

Swagger sirvió mucho durante el desarrollo, porque con Swagger UI se pudieron probar todos los endpoints sin escribir un cliente ni usar Postman.

Un monolito como este es fácil de construir y de desplegar, porque todo es un solo proyecto y un solo ejecutable. Para una aplicación pequeña como esta es suficiente; si el sistema creciera, la división en capas facilitaría separar partes más adelante.

5.2 Lecciones aprendidas

No todos los motores de base de datos manejan los mismos tipos. El modelo venía pensado para MySQL y en SQL Server hubo que reemplazar ENUM por un CHECK y quitar el ancho de los INT.

Las llaves compuestas complican las plantillas que genera Visual Studio, porque estas asumen un solo id. Fue necesario cambiar las rutas, los enlaces y los formularios de estudios a mano.

Las propiedades de navegación de EF Core pueden causar ciclos al convertir a JSON y errores de validación en los formularios. Conviene revisar las entidades generadas antes de exponerlas en la API.

Las llaves foráneas sin cascada impiden borrar registros que tienen dependientes. Hay que decidir si el borrado se maneja en la base de datos o en el código; aquí se hizo en los repositorios.

Tener la cadena de conexión en appsettings.json y no en el código facilita correr el proyecto en otro equipo, donde la instancia de SQL Server puede llamarse distinto.

6. Referencias

Alur, D., Crupi, J. y Malks, D. (2003). Core J2EE Patterns: Best Practices and Design Strategies (2.ª ed.). Prentice Hall.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Krasner, G. E. y Pope, S. T. (1988). A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. Journal of Object-Oriented Programming, 1(3), 26-49.

Microsoft. (s.f.). Información general de ASP.NET Core MVC. Microsoft Learn. https://learn.microsoft.com/es-es/aspnet/core/mvc/overview

Microsoft. (s.f.). Entity Framework Core. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/

Microsoft. (s.f.). Scaffolding (ingeniería inversa). Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/managing-schemas/scaffolding/

Microsoft. (s.f.). Introducción a Swashbuckle y ASP.NET Core. Microsoft Learn. https://learn.microsoft.com/es-es/aspnet/core/tutorials/getting-started-with-swashbuckle

Microsoft. (s.f.). Inserción de dependencias en ASP.NET Core. Microsoft Learn. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/dependency-injection

Microsoft. (s.f.). Documentación técnica de SQL Server. Microsoft Learn. https://learn.microsoft.com/es-es/sql/sql-server/

OpenAPI Initiative. (2021). OpenAPI Specification v3.0.3. https://spec.openapis.org/oas/v3.0.3